实验报告

数字逻辑与计算机组成实验

lab08：数据存储器（RV32I 访存）

姓名：　法童

学号：　251250046

邮箱：　251250046@smail.nju.edu.cn

实验时间：　2026-10-30

1 实验目的

1. 理解字节寻址数据存储器的组织方式：128 KiB 容量如何拆成字索引与字内字节偏移，字节、半字、字访问分别看哪几位地址。
2. 掌握 RV32I 访存指令的 MemOP 译码，以及 lb/lh 的符号扩展与 lbu/lhu 的零扩展。
3. 在不使用 IP 核的前提下，用行为级 Verilog 实现同步读、按字节部分写入的数据存储器。
4. 用 16 位拨码开关构造 MemOP、写使能、地址与写数据，用 8 位数码管动态扫描显示 32 位读出数据，完成从代码到上板的整套流程。

2 实验原理

2.1 地址划分与容量

128 KiB 即 2^{17} 字节，按 32 位字组织正好是 32768 个字。地址是字节地址：

- 字索引取 `addr[16:2]`，共 15 位，覆盖 32768 个字；
- 字内字节偏移取 `addr[1:0]`，4 个字节通道；
- 半字偏移取 `addr[1]`，因为半字只会落在偶数字节上。

按手册约定，4 字节访问时地址低 2 位为 00、2 字节访问时最低位为 0，因此不必处理非对齐访问。

2.2 MemOP 编码

MemOP 是 3 位控制信号：低 2 位选访问宽度，最高位在 load 时选择扩展方式。MemOP 与 RV32I 访存指令的对应关系如下。
写操作只会用到 `memop[1:0]`，`memop[2]` 保持 0；读操作中 `memop[2]`=1 表示无符号，即零扩展。

2.3 读写时序与部分写入

读在 `rdclk` 上升沿进行：先把整个 32 位字读出，再按 MemOP 从字中取出 8 位或 16 位并拼接扩展。写在 `wrclk` 上升沿进行，并且只改动 `datain` 对应的字节通道，其余

表 1: MemOP 编码与访存操作

指令	MemOP	宽度	读 / 写处理
`lb`	000	1 字节	符号扩展（复制 `dataout[8*addr[1:0]+7]`）
`lh`	001	2 字节	符号扩展（复制 `dataout[16*addr[1]+15]`）
`lw`	010	4 字节	原样输出
`lbu`	100	1 字节	高位补 0
`lhu`	101	2 字节	高位补 0
`sb`	000	1 字节	只写 `datain[7:0]` 到对应字节通道
`sh`	001	2 字节	只写 `datain[15:0]` 到对应半字通道
`sw`	010	4 字节	整个字写入

字节保持原值——这正是字节寻址存储器的“字节写使能”语义。行为级实现用带基址的索引部分选择

```
ram[addr[16:2]][8*addr[1:0] +: 8] = datain[7:0];
```

基址可变、宽度必须为常量，等价于硬件里按字节选通的写使能。读和写使用独立的时钟。

2.4 数码管动态扫描与按键消抖

Nexys A7 的八位数码管为共阳极，段选与位选都是低有效。8 位数字不能同时点亮，用 1 kHz 的扫描节拍每次只选通一位，8 ms 扫完一轮，刷新率约 125 Hz，靠视觉暂留看到完整读数；最左边一位显示 `dataout[31:28]`，最右边一位显示 `dataout[3:0]`，小数点不亮。

读、写时钟由按键提供，方便逐拍手动测试。机械按键有抖动，若直接把按键电平当作时钟会一次按下产生多个边沿，所以先做消抖：用 10 ms 节拍连续采样两次都一致才认为按键稳定，再对稳定电平取上升沿，输出只高一拍的脉冲作为 `rdclk`/`wrclk`。

3 实验环境与器材

- 开发板：Nexys A7-100T（FPGA 型号 xc7a100tcsg324-1）
- 开发工具：Vivado 2020.2（非工程模式，`build.bat` / `prog.bat` 脚本驱动）
- 硬件描述语言：Verilog HDL

4 设计思路

4.1 系统结构

整个工程由时基、按键消抖、数据存储器和显示四部分组成。

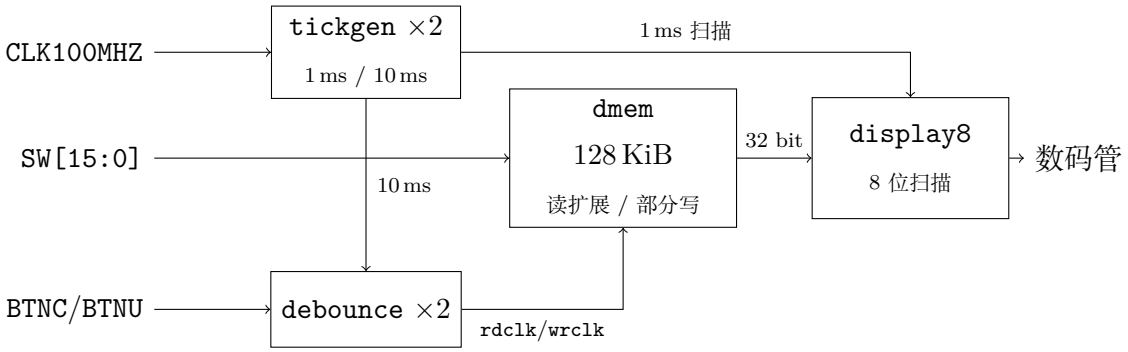


图 1: 数据存储器上板工程的系统框图

表 2: 板级端口分配

板级资源	顶层信号	功能
SW[2:0]	memop	存储器操作类型
SW[3]	we	写使能，1 允许写入
SW[7:4]	addr[3:0]	地址低 4 位，其余高位补 0
SW[15:8]	datain	8 位写数据，重复拼接 4 次成 32 位
BTNC	rdclk	读时钟（消抖后单周期脉冲）
BTNU	wrclk	写时钟（消抖后单周期脉冲）
CLK100MHZ	CLK100MHZ	时基与扫描时钟
AN[7:0]、CA..CG、DP	数码管	32 位十六进制显示，小数点熄灭

4.2 数据存储器

存储体是一个 32768 深的 32 位数组，读写各占一个时钟块。读操作先取出整字，再按 MemOP 抽取并拼接扩展位：

```
always @(posedge rdclk) begin
    dataout = ram[addr[16:2]];
    case (memop)
        3'b000: dataout = {{24{dataout[8*addr[1:0]+7]}}, dataout[8*
            addr[1:0] +: 8]};
```

```
        3'b001: dataout = {{16{dataout[16*addr[1]+15]}}, dataout[16*
            addr[1] +: 16]};
        3'b100: dataout = {24'b0, dataout[8*addr[1:0] +: 8]};
        3'b101: dataout = {16'b0, dataout[16*addr[1] +: 16]};
        default: ;
    endcase
end
```

写操作按 memop[1:0] 决定写整个字、半字还是字节，索引部分选择把 datain 的低字节、低半字放到地址对应的字节通道：

```
always @(posedge wrclk)
    if (we) begin
        case (memop[1:0])
            2'b00: ram[addr[16:2]][8*addr[1:0] +: 8] = datain[7:0];
            2'b01: ram[addr[16:2]][16*addr[1] +: 16] = datain[15:0];
            2'b10: ram[addr[16:2]] = datain;
            default: ;
        endcase
    end
```

4.3 顶层映射与显示

顶层把 16 位开关拆成四组，写数据在 HDL 里重复 4 次拼成 32 位：

```
dmem mymem(
    .addr   ({28'b0, SW[7:4]}),    // 地址低 4 位，高位补 0
    .dataout(dout),
    .datain ({4{SW[15:8]}}),       // 8 位数据重复拼接 4 次
    .rdclk  (rdclk),
    .wrclk  (wrclk),
    .memop  (SW[2:0]),
    .we     (SW[3])
);
```

显示部分用 3 位计数器在 8 位数字之间循环，把当前位的 4 位数据送给 hex7seg 译码，同时拉低对应的位选：

```
reg [2:0] select = 3'd0;
always @(posedge clk)
```

```
    if (tick) select <= select + 3'd1;

case (select)
    3'd0: nibble = data[3:0];     // 最右位
    3'd1: nibble = data[7:4];
    // ... 依次为 data[11:8] 到 data[27:24]
    3'd7: nibble = data[31:28];   // 最左位
endcase

case (select)
    3'd0: seg7 = 8'b11111110;     // AN[0] 低有效
    3'd1: seg7 = 8'b11111101;
    // ...
    3'd7: seg7 = 8'b01111111;
endcase
```

5 实验过程

1. 编写 dmem.v：声明 32768 个 32 位存储单元，实现上升沿读整字并按 MemOP 抽取扩展，上升沿按字节部分写入。
2. 复用 lab04 的 tickgen（1 ms 扫描节拍、10 ms 消抖节拍）、debounce 与 hex7seg，新写 display8 完成 8 位扫描与位选译码。
3. 编写 top.v，把 SW[15:0] 拆成 MemOP、写使能、地址与写数据，把两个按键消抖后的脉冲接到 rdclk/wrclk。
4. 编写约束文件 dmem.xdc：从根目录 nexysa7.xdc 复制，只取消本次用到引脚的注释（CLK100MHZ、SW[15:0]、BTNC、BTNU、CA..CG、DP、AN[7:0]）；其中 SW[8]、SW[9] 属于 bank 34，保持 LVCMOS18 不变。
5. 运行 build.bat lab08\dmem 完成综合、实现并生成比特流 build/top.bit；运行 prog.bat lab08\dmem 下载到开发板，按下面的测试步骤拨开关、按键验证。

6 测试方法

如何验证：以上板实测验证。用拨码开关设定 MemOP、写使能和地址，用 8 个开关给写数据，按一次 BTNU 写入、再按一次 BTNC 读出，从数码管读数对照预期。

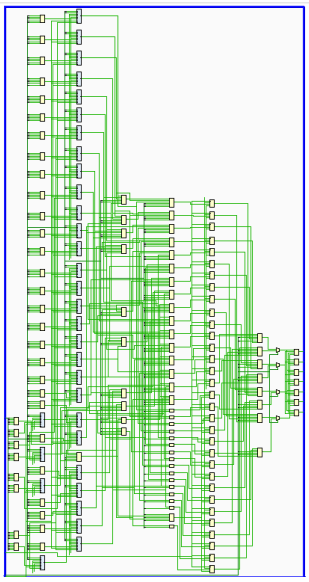


图 2: 综合、实现与比特流生成完成

测试信号的选择：

- **写类型覆盖三种宽度**：`sw`（010）、`sb`（000）、`sh`（001）各做一次；
- **读类型覆盖字、字节与两种扩展**：`lw`（010）、`lb`（000）、`lbu`（100）各读一次，`lb` 与 `lbu` 读同一个地址，直接对比符号扩展与零扩展；
- **地址覆盖同一个字内的字节通道**：先在地址 0 写整个字，再分别在地址 1 写字节、地址 2 写半字，每次都读回整个字，检查只有对应字节被改写；
- **写数据选符号位为 1 的值**：字节写用 0x80，`lb` 读出 FFFFFF80、`lbu` 读出 00000080，两种扩展的差别一眼可见；
- **先写后读成对比较**：每组写操作之后都读一次，避免只验证写或只验证读。

开关值依次按 `SW[15:8]`、`SW[7:4]`、`SW[3]`、`SW[2:0]` 排列：

7 实验结果

7.1 上板

按上面的测试步骤下载比特流后逐组操作，数码管读数与预期一致：

- 写 0xA5 到地址 0 后，字读显示 A5A5A5A5，8 个数码管同时稳定点亮，无可见闪烁；

表 3: 上板测试步骤

步骤	开关设置	按键	预期显示
写 0xA5 到地址 0	A5 0 1 010	BTNU	—
字读地址 0	A5 0 0 010	BTNC	A5A5A5A5
字节写 0x80 到地址 1	80 1 1 000	BTNU	—
字读地址 0	80 0 0 010	BTNC	A5A580A5
字节读地址 1（lb）	80 1 0 000	BTNC	FFFFFF80
字节读地址 1（lbu）	80 1 0 100	BTNC	00000080
半字写 0x12 到地址 2	12 2 1 001	BTNU	—
字读地址 0	12 0 0 010	BTNC	121280A5

- 字节写 0x80 到地址 1 后，字读显示 A5A580A5，说明只有地址 1 对应的字节被改写；同一地址用 lb 读出 FFFFFF80、用 lbu 读出 00000080，符号扩展与零扩展的区别清晰可见；
- 半字写 0x12 到地址 2 后，字读显示 121280A5，地址 0 与 1 的字节保持原值；
- 按键每按一次只产生一个读写脉冲，显示只变化一次，没有出现重复写入或跳变。

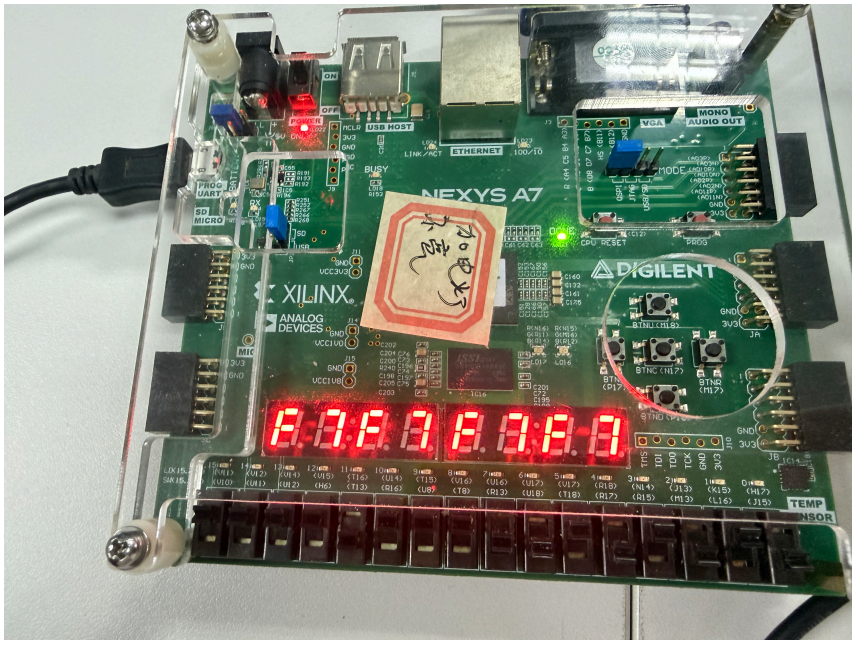


图 3: 上板实测：sw 写入后 lw 读回 A5A5A5A5

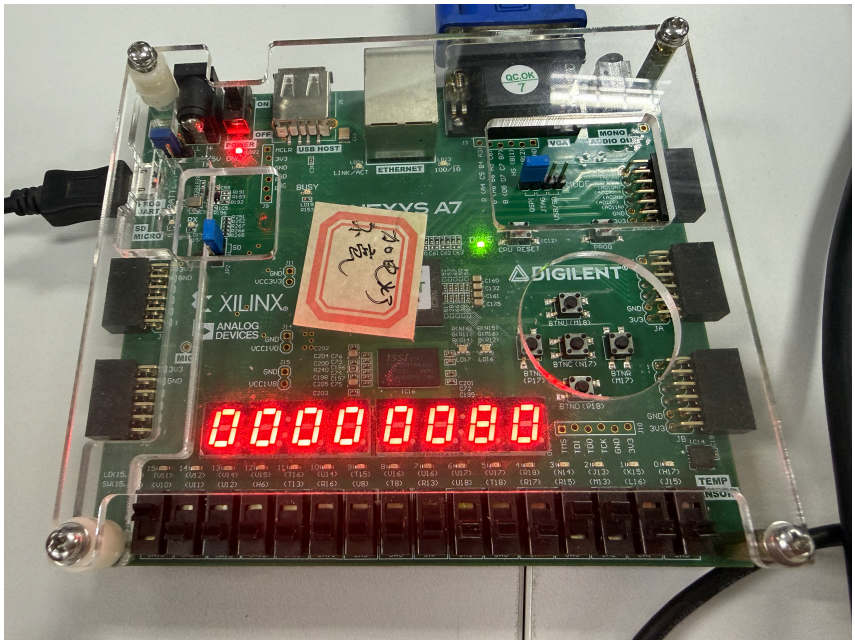


图 4: 上板实测：sb 后 lb / lbu 分别为 FFFFFF80 / 00000080

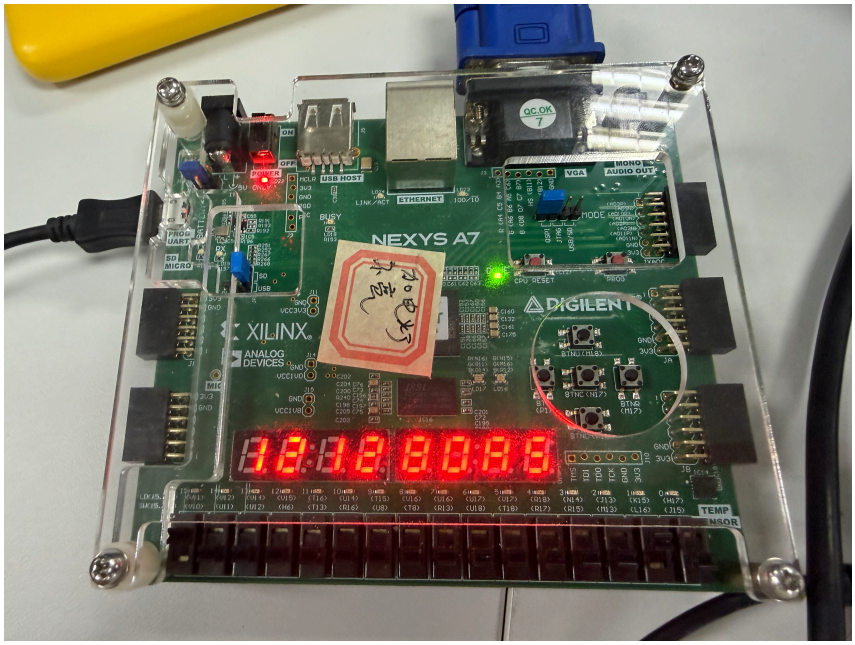


图 5: 上板实测：sh 后 lw 读出 121280A5

8 问题与解决办法

- **可变宽度的部分选择语法错误**：最初写 `dataout[addr[5:0]+7:addr[5:0]]`，Vivado 报 `range must be bounded by constant expressions`，模块被忽略。原因是普通部分选择 `[msb:lsb]` 的两端必须是常量，宽度不能随地址变化。解决：改用索引部分选择 `[base +: width]`——基址可以是变量，宽度写成常量 8 或 16，读写两端统一用这种写法。

- **地址切位与容量对不上**：原来按“64 字节一个存储单元”去取 `addr[17:6]` 和 `addr[5:0]`，但存储体是 32 位字，128 KiB 只有 32768 个字。这样每个字的索引都偏了 4 位，不同地址会落到同一个单元。解决：先由容量算出字索引为 `addr[16:2]`、字节偏移为 `addr[1:0]`、半字偏移为 `addr[1]`，读写的地址切位保持一致。

- **符号扩展方向写反**：一开始把 `memop[2]` 当作“符号扩展”标志（为 1 时复制符号位），与手册的约定相反——`memop[2]`=1 对应 lbu/lhu，应当高位补 0。解决：按 MemOP 直接分情况，000/001 复制符号位，100/101 高位补 0；测试数据专门取符号位为 1 的值（如 0x80）把两种情况区分开。

- **display8 的输出接到子模块输出上**：`output reg [6:0] h` 既在端口声明里是 `reg`，又直接连到 `hex7seg` 的输出，综合报 `variable 'h' should not be used in output port connection`。原因是一个变量不能同时被过程赋值和模块输出驱动。解决：把 `h` 改成 `output wire [6:0]`，由 `hex7seg` 实例直接驱动。

9 启示与建议

启示：

1. 字节寻址存储器的核心是“部分写入”：读的时候先取整字再抽取扩展，写的时候只改选中通道、其余字节保持原值。`[base +: width]` 这种写法在行为级正好对应硬件里的字节写使能，读写两端的地址切位必须完全一致，否则读回的值和写进去的对不上。

2. 这次一开始按 64 字节单元去切地址，即使把语法问题改掉，字索引也会错位、不同地址落到同一个字。写代码前要先把“容量—字宽—偏移”三者的关系算清楚。

建议：现在地址只接了低 4 位，只能覆盖 16 字节，字索引高位的进位和跨字访问都测不到；如果多引几位地址开关（或者加一个“地址加一”的按键），就能把整个 128 KiB 空间都走一遍，也更容易发现地址切位一类的问题。